

Flush Equivalence Determines Interleaved Behavior for Stateless PIFO Trees

Definitions

Definition 1 (PIFO). An entry with values in a set X is a triple

$$(x, \rho, \tau) \in X \times \mathbb{N} \times \mathbb{N},$$

where ρ is its rank and τ is its arrival stamp. A PIFO is a finite queue of entries. A pop removes the entry with lexicographically least key (ρ, τ) : lower ranks come first, and equal ranks are served in FIFO order. A push appends its entry to the queue.

Definition 2 (PIFO tree and annotated path). A PIFO-tree topology is either a leaf or an internal node with a finite ordered list of child topologies, possibly empty. A state places a PIFO at every node. A leaf PIFO contains packets, while an internal PIFO contains child indices.

An annotated root-to-leaf path specifies, at every internal node, a child index and the rank of the reference inserted there, and specifies the packet's rank at the final leaf. Pushing a packet with arrival stamp τ inserts entries carrying that same stamp at every node on the path. A path is valid when its shape and all its child indices fit the topology. On a path mismatch the formal push operation leaves the mismatched subtree unchanged; this clause is unreachable for the valid schedulers considered below.

To pop a tree, pop its root PIFO and, at an internal node, recurse into the selected child. The operation is atomic: it either reaches a leaf and returns a packet together with the updated tree, or returns **none** and leaves the original state unchanged.

Definition 3 (Stateless scheduler and observations). For $k \in \mathbb{N}$, let $A = \text{Fin}(k)$. A stateless scheduler S on A consists of a PIFO-tree topology and one fixed annotated path $S(a)$ for each $a \in A$. Every push of type a uses this same path, independently of the state and arrival time. The scheduler is *valid* if every path $S(a)$ is valid.

An operation is either $\text{push}(a)$ or pop . A run starts with every queue empty and stamps successive pushes $1, 2, \dots$. Its observation, denoted $\text{run}_S(u)$, has one item for every pop in the operation word u : it records **some**(a) when type a is emitted and **none** when the pop fails. In the latter case the state is unchanged.

Definition 4 (Flush and interleaved equivalence). For a packet word $w = a_1 \cdots a_n$, put

$$\text{flushOps}(w) = \text{push}(a_1) \cdots \text{push}(a_n) \text{pop}^n.$$

Schedulers S, T are *flush-equivalent*, written $S \simeq_{\text{flush}} T$, when

$$\text{run}_S(\text{flushOps}(w)) = \text{run}_T(\text{flushOps}(w)) \quad \text{for every } w \in A^*.$$

They are *interleaved-equivalent*, written $S \simeq_{\text{inter}} T$, when

$$\text{run}_S(u) = \text{run}_T(u) \quad \text{for every operation word } u,$$

with pushes and pops arbitrarily interleaved and over-popping observable.

Theorem 1. For every $k \in \mathbb{N}$ and every pair of valid stateless schedulers S, T over $\text{Fin}(k)$,

$$S \simeq_{\text{flush}} T \implies S \simeq_{\text{inter}} T.$$

Equivalently, with the definitions expanded, for all k, S, T ,

$$\begin{aligned} \text{Valid}(S) \longrightarrow \text{Valid}(T) \longrightarrow \\ (\forall w, \text{run}_S(\text{flushOps}(w)) = \text{run}_T(\text{flushOps}(w))) \longrightarrow \\ (\forall u, \text{run}_S(u) = \text{run}_T(u)). \end{aligned}$$

Proof. We establish the equivalent assertion for every finite alphabet A ; renaming its elements along $A \cong \text{Fin}(|A|)$ then proves the theorem. Our notation is as follows. The projection of a word to $B \subseteq A$ is $w|_B$. For a natural-rank total preorder r , $\text{sort}_r(w)$ means the stable rank-sort of w , and $\pi_P(w)$ is the word of block names obtained from a partition P .

Operational invariants. Fix a child c of an internal node. In any state reachable under valid paths,

$$\#\{\text{references to } c\} = \#\{\text{packets in the subtree below } c\}. \quad (1)$$

Indeed, the two counts start at zero. A valid push routed through c adds one to each, and a completed pop routed through c removes one from each. Equation (1) follows by induction on the run. In particular, whenever an internal reference wins, its target contains a packet. The recursive failure arm of an atomic pop is therefore never taken in a reachable state. A pop returns **none** precisely at total packet count zero, making the success/failure pattern a function of the operation word alone.

Only the order of stamps enters a queue comparison. Thus any order-preserving relabeling of the stamps leaves the transition sequence unchanged. Relabeling a sparse increasing sequence by $1, 2, \dots$ will be called *timestamp compression*; it lets us regard a driven child run with global gaps as a standalone run.

For bookkeeping, give every enqueued entry the identity of the push that created it. These ghost identities neither affect a comparison nor alter the untagged run. Stamps are injective separately in each queue, and also in every pending list used later. Of course one push produces same-stamped entries at different tree levels; the assertion is only queue-local. Consequently a selector never has two entries with the same full key (rank, arrival).

The two-type calculation. Retain only two distinct types x, y and compare their fixed routes. Along the common prefix, every queued value is the same child index. Whatever reference is removed there, the node simply asks that child for one packet, so no type-level choice is made on this prefix. The first route split, if there is one, is different: choosing a child chooses between an x -only branch and a y -only branch. If the routes never split, their common leaf makes the choice directly. Collapsing the irrelevant prefix and the one-type suffixes shows that the restriction is exactly one stable PIFO with an effective comparison

$$e(x, y) \in \{x < y, x = y, y < x\}.$$

Writing F for its flush output, testing the two possible arrival orders determines which of the three comparisons holds:

$e(x, y)$	$F(xy)$	$F(yx)$
$x < y$	xy	xy
$x = y$	xy	yx
$y < x$	yx	yx

(2)

The two columns are both needed: the middle row is the FIFO-equality case.

Colored-PIFO congruence. Consider two selectors on the same pushed occurrences, ordered respectively by total preorders p and q , and color each type by γ . Assume exact agreement whenever the colors differ:

$$\gamma(x) \neq \gamma(y) \implies \text{cmp}_p(x, y) = \text{cmp}_q(x, y). \quad (3)$$

Equality in (3) is part of the assumption. We claim that the two selectors produce identical color sequences under any common sequence of pushes and pops.

Within one color, group types that have the same comparison profile against all other colors:

$$x \sim y \iff \gamma(x) = \gamma(y) \text{ and } \text{cmp}_p(x, z) = \text{cmp}_p(y, z) \text{ for every } z \text{ of another color.}$$

Condition (3) makes this the same equivalence relation for q . Totality supplies the following facts. First, two distinct profile classes of one color have a common strict ordering under p and q . Second, if a class ties anything of another color, every member ties that witness and all other members under both orders; call the class *pinned*. Third, for any two classes of different colors, their comparison is independent of the chosen representatives and is identical for p and q .

The coupling records, for each profile class, equal pending cardinalities. For a pinned class it records more: the two sides have exactly the same arrival-ordered list of ghost occurrences. These assertions hold initially and after a common push. Consider a pop, and let K_p, K_q be the classes containing the two chosen occurrences. If the classes differ, the count invariant ensures that a K_q -occurrence is available to the p -selector and a K_p -occurrence to the q -selector. Optimality then implies

$$K_p \leq_p K_q, \quad K_q \leq_q K_p,$$

with either inequality valid for all representatives. The classes cannot be two profiles of one color, since their strict ordering is common to both preorders. They therefore have different colors, and the constant cross-class comparison forces a tie.

The tie makes K_p and K_q pinned, so their invariant lists put both chosen occurrences in both queues. Their rank parts are equal on either side. If the choices are o_p, o_q , minimality first gives

$$\text{arr}(o_p) \leq \text{arr}(o_q) \text{ and then } \text{arr}(o_q) \leq \text{arr}(o_p).$$

The arrivals coincide, and queue-local injectivity says $o_p = o_q$, which is impossible for two different classes. Hence both selectors choose one profile class and emit one color. Counts remain matched. When that class is pinned, each selector removes the earliest occurrence from the same list, preserving the stronger clause. An empty pop is simultaneous because the total counts match. The claim follows.

If a use of the lemma is phrased on a subalphabet, it can instead be made global by padding ranks. Put every excluded type at one shared rank above all retained ranks in both orders. The hypothesis then follows by checking inside-inside, the two inside-outside orientations, and outside-outside.

Flush decompositions. Suppose $|A| > 1$. Peel away the initial chain on which every assigned path names one common child. Such a node reveals no routing information: each successful request to it becomes one request to that sole active child, irrespective of the order among its references. The chain ends either at the first genuine branch or at a leaf. In the leaf case, simulate its queue by a new selector carrying the leaf's ranks and route each type to its own one-type FIFO child. Its root queue mirrors the old leaf queue occurrence for occurrence, and selecting a reference calls the corresponding one-type child.

We may therefore replace a scheduler by an online-equivalent object of the form

$$M = (P, r, (M_B)_{B \in P}), \quad (4)$$

and reuse the name M for the replacement. Here P consists of the nonempty type sets routed to the active children, r is the root-rank preorder, and M_B is the scheduler below block B . The partition is *proper*: it has at least two nonempty blocks, so every $B \in P$ is a strict subset of A .

Let $F_M(w)$ be the emitted type word after pushing all of w and draining. For $B \in P$,

$$F_M(w)|_B = F_{M_B}(w|_B) = F_M(w|_B). \quad (5)$$

At the start of the drain, child B has received precisely the projected input $w|_B$. The parent stores one B -reference for each of those pushes, so it requests $|w|_B|$ answers from that child. Removing the traffic of the other blocks changes neither this local input nor these requests, proving both equalities. On the other hand, forgetting the types and retaining only their root destinations gives

$$\pi_P(F_M(w)) = \pi_P(\text{sort}_r(w)). \quad (6)$$

This is simply the stable ordering of the root entries by r .

We say that (P, r) decomposes F when these two identities hold. They imply *block autonomy*. Namely, if the input uses only B , the pattern identity keeps the output in B , while (5) identifies that output with the child computation. For unrestricted w , they also give the projection rule $F(w)|_B = F(w|_B)$.

The meet-order lemma. The central claim is

$(P, r) \text{ and } (Q, s) \text{ decompose } F$ $\implies \exists \text{ a total preorder } t \text{ such that}$ $\text{cmp}_t = \text{cmp}_r \text{ across distinct } P\text{-blocks,} \quad \text{cmp}_t = \text{cmp}_s \text{ across distinct } Q\text{-blocks.}$	(7)
--	-----

Let

$$R = P \wedge Q = \{ B \cap C : B \in P, C \in Q, B \cap C \neq \emptyset \}.$$

For a cell $D = B \cap C$, the two decomposition identities give

$$\begin{aligned} F(w)|_D &= (F(w)|_B)|_C \\ &= F(w|_B)|_C \\ &= F((w|_B)|_C) \\ &= F(w|_D). \end{aligned} \quad (8)$$

Thus restriction to any $D \in R$ commutes with F .

Take x, y from different cells and define $e(x, y)$ by the binary table (2). If their P -blocks differ, the P -normal form routes their references to different root children. On inputs using just x, y , those two destinations determine the emitted type, so the table reads exactly the root comparison:

$$\text{cmp}_e(x, y) = \text{cmp}_r(x, y). \quad (9)$$

The same reasoning with the Q -normal form gives, whenever their Q -blocks differ,

$$\text{cmp}_e(x, y) = \text{cmp}_s(x, y). \quad (10)$$

When both coordinates differ, the single observable value $e(x, y)$ therefore makes the two root comparisons coincide.

We next check triples of meet cells. Plot a cell $B \cap C$ at row B and column C , and select one type from each of three plotted cells. Three different rows put every pair under r , and three different columns put every pair under s , so those cases are already transitive. If neither occurs, the three occupied squares are the two arms and corner of an L. With the corner called x , restricting the two partitions to the triple yields

$$\{x, y\} \mid \{z\}, \quad \{x, z\} \mid \{y\}. \quad (11)$$

Only transitivity can now fail. If it did, the types could be named so that

$$a \leq_e b, \quad b \leq_e c, \quad c <_e a.$$

Use a, b, c as both the insertion order and the ensuing flush input. The three stamps are different, so arrival order decides every weak root tie. It suffices to display the orientation in which one partition groups $\{b, c\}$ and the other groups $\{c, a\}$. On the first root, the two separated pairs force

$$\rho_c < \rho_a \leq \rho_b.$$

Thus the root asks for the blocks in the order $\{b, c\}, \{a\}, \{b, c\}$; if $\rho_a = \rho_b$, the stamp of a is earlier. The paired child saw b before c , and $b \leq_e c$, so its two answers are b, c . The resulting type word is

$$b, a, c.$$

At the other root the forced inequalities are $\sigma_a \leq \sigma_b \leq \sigma_c$. Its block requests, in order, are

$$\{c, a\}, \{b\}, \{c, a\}.$$

The paired child received a first, but the strict comparison $c <_e a$ makes its answers c, a , producing

$$c, b, a.$$

These outputs cannot both equal $F(abc)$, already because their initial letters differ. Rotating a, b, c moves the grouped edge through its other two positions; reversing the roles of the partitions covers the three opposite orientations. Hence e restricts to a total preorder on every choice of three cells.

Encode the cross-cell comparisons in a digraph G . Insert an arc $x \rightarrow y$ whenever $x \leq_e y$, and flag it when $x <_e y$. Consistency amounts to showing that no cycle contains a flagged arc. Put a hypothetical flagged arc first and call it $a \rightarrow b$; we argue by strong induction on the cycle length. If an internal vertex repeats, deleting the intervening closed segment leaves a shorter flagged cycle, so assume the cycle is simple.

At length 2, the return arc $b \rightarrow a$ is incompatible with $a <_e b$. At length 3, every pair of vertices is cross-cell, and the triple preorder excludes a triangle with a flagged side.

For a length-4 cycle (a, b, c, d) , first suppose a, c are in distinct cells. The total preorder on a, b, c , together with $a <_e b \leq_e c$, makes the chord $a \rightarrow c$ flagged; it closes the flagged triangle a, c, d . If instead b, d are in distinct cells, the total preorder on b, c, d supplies $b \rightarrow d$, and a, b, d is a triangle whose flagged side is $a \rightarrow b$. If both diagonals are intra-cell, the vertices alternate between two cells. Either P distinguishes those cells, in which case all four inequalities are read from r , or it does not and they are all read from s . That single rank function cannot satisfy

$$\rho_a < \rho_b \leq \rho_c \leq \rho_d \leq \rho_a.$$

Now let the length be at least 5, with c, d the next two vertices after b . If a, c are in distinct cells, the three-cell result makes $a \rightarrow c$ flagged, and replacing $a \rightarrow b \rightarrow c$ by this chord gives a shorter flagged cycle. If b, d are in distinct cells, the three-cell result supplies $b \rightarrow d$; replacing $b \rightarrow c \rightarrow d$ preserves the flagged head $a \rightarrow b$ and again shortens the cycle. Suppose both diagonals are intra-cell. Then a, d are in distinct cells, since otherwise a, c, d would lie in one cell while the consecutive pair c, d must not. If the chord has $d \rightarrow a$ (including a tie), it closes the already excluded flagged four-cycle (a, b, c, d) . Otherwise the same r -or- s rank function governs all four cross-cell comparisons, and

$$a <_e b \leq_e c \leq_e d;$$

hence $a \rightarrow d$ is flagged. Replacing the first three edges by this chord produces a shorter flagged cycle. This completes the induction and proves the claim.

Let $z \rightsquigarrow x$ denote reachability in this graph, including the length-zero path, and define the natural rank

$$t(x) = \#\{z : z \rightsquigarrow x \text{ and } x \not\rightsquigarrow z\}. \quad (12)$$

Along any edge $x \rightarrow y$, the strict-predecessor set counted at x is contained in the one counted at y . If the edge is flagged, y cannot reach x , for that would make a flagged cycle; consequently x is a new strict predecessor of y , and $t(x) < t(y)$. A tied comparison supplies edges in both directions and gives equal ranks. Thus the numeric preorder defined by t realizes every cross-cell constraint. By (9)–(10),

$$\text{cmp}_t = \text{cmp}_r \text{ across distinct } P\text{-blocks}, \quad \text{cmp}_t = \text{cmp}_s \text{ across distinct } Q\text{-blocks}. \quad (13)$$

This proves the meet-order lemma.

Strong induction on the alphabet. Induct simultaneously over the cardinality of a finite alphabet. With no types, all pops fail. With one type, packet count decides whether a pop succeeds, and there is only one possible successful value.

Now take $|A| > 1$ and flush-equivalent S, T . Replace both schedulers by the online-equivalent forms supplied above, keep the names S, T for those replacements, and write

$$S = (P, r, (S_B)_{B \in P}), \quad T = (Q, s, (T_C)_{C \in Q}).$$

Their flush transformer is one function F . Apply the meet-order lemma to obtain $R = P \wedge Q$ and t as in (13).

Change S 's root ranks from r to t , leaving all children fixed. With a block of P treated as a color, the colored congruence hypothesis is exactly (13). The old and new roots consequently call the same blocks at the same pops, which drives identical operation words into corresponding children. If S^t denotes the changed scheduler,

$$S \simeq_{\text{inter}} S^t \quad \text{and} \quad F_{S^t} = F.$$

The same operation on T gives

$$T \simeq_{\text{inter}} T^t \quad \text{and} \quad F_{T^t} = F.$$

Both changed schedulers still flush to F . Their root patterns now use the same preorder, so (6) says simultaneously that

$$\pi_P(F(w)) = \pi_P(\text{sort}_t(w)), \quad \pi_Q(F(w)) = \pi_Q(\text{sort}_t(w)). \quad (14)$$

For each $D \in R$, fix once and for all a valid scheduler U_D whose flush function on D is $F|_D$. Such a scheduler exists: restrict S to D and use (8). Given $B \in P$, build V_B by placing a t -selector above those U_D with $D \subseteq B$. We show that this replacement has the same flush function as S_B :

$$F_{S_B} = F_{V_B}. \quad (15)$$

To identify two B -words, it is enough to know their word of Q -block labels and, separately, their subsequence in every Q -block. Indeed, a left-to-right scan of the labels uniquely interleaves those subsequences. For $v \in B^*$, autonomy identifies $F_{S_B}(v)$ with $F(v)$; its Q -labels are those of $\text{sort}_t(v)$ by (14). The root of V_B gives exactly those labels as well, since its children are the nonempty cells $B \cap C$, $C \in Q$.

For the remaining subsequences, choose $C \in Q$ and set $D = B \cap C$. When D is empty, both outputs are B -valued and have empty C -projection. Otherwise,

$$\begin{aligned} F(v)|_C &= F(v|_C) \\ &= F(v|_D) \\ &= F_{U_D}(v|_D) \\ &= F_{V_B}(v)|_C. \end{aligned} \quad (16)$$

The four steps use, in order, Q -block autonomy, the fact that v is a B -word, the defining choice of U_D , and the child projection formula for V_B . Thus label patterns and every labeled subsequence match, proving (15).

Properness of P gives $|B| < |A|$, so the induction hypothesis converts the flush equality into

$$S_B \simeq_{\text{inter}} V_B. \quad (17)$$

Perform these child replacements simultaneously below the unchanged root of S^t . Each B -child sees the pushes projected to B , and it receives a pop exactly when the root emits a B -token. Equation (17) therefore preserves the composite run. The stamps in this driven word are sparse global stamps; compressing them supplies the consecutive stamps assumed by its standalone induction instance.

There are now two consecutive t -selectors. Define U by removing the middle level and letting one t -root route directly to the already fixed children U_D . Let Ω be the pending source pushes, using their ghost identities. The coupling invariant can be recorded compactly as

$$\begin{aligned} \text{state of each two-level } U_D &= \text{state of the corresponding one-level } U_D, \\ \text{occurrences at the outer selector} &= \Omega \quad (\text{labeled by } P), \\ \text{occurrences at } U\text{'s root} &= \Omega \quad (\text{labeled by } R), \\ \text{occurrences at the inner selector for } B &= \Omega|_B \quad (\text{labeled by } R). \end{aligned}$$

Each displayed queue orders occurrences by the identical pair $(t, \text{arrival})$, whose second component is injective. Suppose the outer level chooses o in block B . It is the sole minimum of Ω ; hence it is also the sole minimum of the restricted set $\Omega|_B$. The inner level and U 's root consequently choose the same ghost occurrence o , and both invoke the same state U_D . On success, all corresponding queues delete o . On a failure below them, atomicity restores both composite states. A push adds the corresponding entry to each listed queue. The invariant is preserved, proving the collapse. Chaining it with the rank swap and child substitutions gives

$$S \simeq_{\text{inter}} U. \quad (18)$$

Apply the mirror construction to T , crucially retaining the same chosen family (U_D) . This time the reconstruction takes block labels from the P -part of (14) and its projected words from the S -decomposition. Each $C \in Q$ is smaller than A , so induction and the identical collapse yield

$$T \simeq_{\text{inter}} U. \quad (19)$$

Both original schedulers are therefore interleaved-equivalent to U , and hence to one another. Reinsert the failed-pop positions: by (1) these positions depend only on the input operation word. The resulting equality is equality of the exact lists produced by run, including every `none`. $\square$

Kernel-witness scope. The colored-PIFO coupling is witnessed by `KernelB.lean` (`Rel`, `Rel_pop`) and `Kernel.lean` (`tie_pins`, `strict_of_sameColor`, `memInd`); rank padding is witnessed by `Refine2A.lean` (`rankBound`, `padK`, `crossAgree_pad`) and `Refine2B.lean` (`rel_run_split`). The three-cell calculation is witnessed by `Ultra2Flush.lean` (`flush3_first`, `flush3_mid`, `flush3_last`, `cfg1`, `cfg2`, `cfg4`, `composite_noCycle`, and `composite_TripleTrans`). Rank padding is not used in the main argument above. The fixed-head marked-cycle construction, including its counting linearization, is checked by `KConstruct.lean` (`no_chain`, `triangle`, `K`, `K_iff`) only for flat normal forms. The arbitrary-composite extension is instead witnessed by `Ultra2Extend.preorder_extension` and `Ultra2Flush.composite_extension`, through a different walk-normalization proof.

In `Ultra2Main.lean`, the root swap is witnessed by `step` and `rankSwap_interEquiv`. The reconstruction witnesses are `block_two_level`, `block_flush_eq`, and `reconstruct`; the strict-key collapse witnesses are `tmin_strict` and `collapse_runFrom`, using the pending-list invariant `TSorted`. The compiled capstone derives the two cell families separately, collapses each side, and then uses cellwise induction; it does not literally use the preselected common family (U_D) in the prose organization above. Finally, `Stamped.lean` witnesses the alternative formulation with an explicit arbitrary stamp on every push and `sInterEquiv` quantified over arbitrary stamped words; the proof above instead uses timestamp compression.